به نام خدا

ساختمان داده‌ها و الگوریتم‌ها (۴۰۲۵۴)

دانشگاه صنعتی شریف — تمرین اوّل

مدرس: مهدی صفرنژاد — انتشار: ۱ اسفند ۹۸

تفکر الگوریتمی

زمان تحویل این تمرین (شامل بخش نظری و بخش عملی) شنبه ۱۰ اسفند ۹۸ ساعت ۱:۰۰ بامداد (یک ساعت پس از نیمه‌شب جمعه) می‌باشد. به توضیحات سایت درس دربارهٔ تمرین‌ها، تحویل آن و تأخیر توجه کنید. تمرین‌ها از سایت کوئرا تحویل گرفته می‌شوند. قسمت‌های نظری به صورت فایل PDF تحویل گرفته می‌شوند که این فایل یا باید با حروف‌چینی رایانه‌ای تولید شده باشد و یا اگر دست‌نوشته است، توسط نرم‌افزارهایی مثل CamScanner یا Office Lens و یا با دستگاه اسکنر تصویربرداری شده باشد.

---

۱ تمرینات تئوری

سؤال ۱. [۱۲ نمره] توابع هر دسته را بر اساس پیچیدگی‌شان مرتب کنید. (فقط ترتیب را از کوچک به بزرگ بنویسید؛ نیاز به نوشتن اثبات نیست.)

آ. [۴ نمره]

$$f_۱(n) = \binom{n}{n-۳} \qquad f_۲(n) = ۱۰۰!$$
$$f_۳(n) = \pi^n \qquad f_۴(n) = n\sqrt[۳]{n}$$

ب. [۴ نمره]

$$f_۱(n) = n\sum_{i=۱}^{n} \frac{۱}{\sqrt{i}+\sqrt{i+۱}} \qquad f_۲(n) = n^۵ \cdot ۵^n$$
$$f_۳(n) = ۲^{\sqrt[n]{n^{n+۱}}} \qquad f_۴(n) = \sqrt{۲^{\sqrt{n}}}$$

پ. [۴ نمره]

$$f_۱(n) = ۲^{\log_۲ n^۲} \qquad f_۲(n) = n\sqrt{n}$$
$$f_۳(n) = n\binom{n}{۲} \qquad f_۴(n) = \sqrt{n}\log n$$

پاسخ: آ. $f_۲ < f_۴ < f_۱ < f_۳$

ب. $f_۱ < f_۴ < f_۲ < f_۳$

پ. $f_۴ < f_۲ < f_۱ < f_۳$

سؤال ۲. [۱۲ نمره] با استفاده از درخت بازگشت، پیچیدگی تابع $T(n)$ را در هر قسمت به دست آورید. نوشتن جواب نهایی کفایت می‌کند و نیاز به توضیح و اثبات نیست.

آ. [۴ نمره] $T(n) = ۲T(n/۲) + \Theta(nlogn)$

ب. [۴ نمره] $T(n) = ۳T(n/۴) + ۵n$

پ. [۴ نمره] $T(n) = ۲T(n-۱) + \Theta(n)$

پاسخ:

آ. $O(n \times log^{۲}n)$

ب. $O(n^{log_{۳}۴})$

پ. $O(۲^n)$

سؤال ۳. [۱۶ نمره] n نقطه روی صفحه مختصات به شنگدباو داده‌شده‌است. او می‌خواهد تعداد مربع‌هایی را بیابد که رئوس آن روی این نقاط باشند. برای این‌کار الگوریتم زیر را پیشنهاد کرده‌است:

هر سری بین تمامی خطوط افقی و عمودی که روی آن حداقل یک نقطه باشد، آن خطی را در نظر می‌گیریم که تعداد نقاط رویش کمینه‌باشد. سپس به ازای هر جفت از نقاط که روی آن خط هستند چک می‌کنیم آیا مربعی هست که این دو نقطه دو سر یکی از اضلاعش باشند. توجه کنید حداکثر دو مربع می‌توانیم داشته‌باشیم که دو نقطه‌اش نقاطی باشند که ما در نظر گرفته‌ایم. پس از اینکه تمامی مربع‌هایی را یافتیم که نقاط رئوسش روی این خط هستند. تمامی نقاط روی این خط را پاک می‌کنیم و به صورت بازگشتی دوباره الگوریتم را تکرار می‌کنیم.

فرض کنید چک کردن اینکه ۴ رأس آیا باهم تشکیل مربع می‌دهند یا نه ۱ واحد هزینه دارد. از این نکته استفاده کنید که خطی افقی یا عمودی وجود دارد که روی آن حداکثر $\sqrt{n}$ تا نقطه‌باشد؛ سپس با استفاده از این نکته ثابت کنید $O(n\sqrt{n})$ واحد هزینه کافیست بپردازیم. سپس از این موضوع نتیجه بگیرید که تعداد مربع‌هاییکه رئوس آن روی این نقاط باشند از همین مرتبه‌است.

پاسخ: فرض کنید تعداد a تا نقطه پیدا کرده باشیم که با هم روی یک خط (عمودی یا افقی) قرار گرفته باشند. باتوجه به اینکه مربع ها افقی یا عمودی هستند مربع‌هاییکه ممکن است از روی این نقاط ساخته شوند $a \times (a-۱)$ تا هستند. یعنی به ازای هر جفت نقطه می‌توانیم مربع‌هاییکه این جفت یک ضلعش را تشکیل دهند چک کنیم. کافیست ببینیم در نقاط متناظر در دو طرف خط آیا مربعی وجود دارد یا خیر. حالا که همه مربع‌هاییکه با این نقاط می‌تواند به وجود بیاید را پیدا کردیم می‌توانیم آن نقاط را حذف کنیم و بقیه مسئله را مشابهاً حل کنیم. بنابراین اگر خطی افقی یا عمودی پیدا کنیم که روی آن مجموعه $|A|$ از نقاط باشد. و S مجموعه همه نقاط‌مان باشد و $T(n)$ برابر تعداد مربع‌های تشکیل شده از این نقاط باشد داریم:

$$T(S) = T(S-A) + a \times (a-۱)$$

حال ثابت می‌کنیم مجموعه A از نقاط وجود دارد که $|A| \leq \sqrt{|S|}$

برای اثبات این موضوع فرض کنید همه خطوط افقی لااقل $\sqrt{|S|}$ تا نقطه دارند. (توجه کنید منظورمان بین خطوطی هست که لااقل یک نقطه رویشان ظاهر شده‌باشد) اگر اینطور نباشد یکی از آن خطوط را با نقاط رویش در نظر بگیریم به ما مجموعه A مورد نظر

را خواهد داد. اما اگر همه از این مقدار بیشتر باشند واضح است که تعداد خطوط افقی‌ای که لااقل $\sqrt{|S|}$ تا نقطه دارند حداکثر $\sqrt{|S|}$ تا اند! (چرا؟)

بنابراین هر خط عمودی‌ای که در نظر بگیریم که حداقل یک نقطه رویش باشد. در نقاط تقاطعش با خطوط افقی ممکن است نقطه دیگری داشته باشند. بنابراین چون می‌دانیم تعداد خطوط افقی در این حالت حداکثر $\sqrt{|S|}$ تاست می‌توان دید که تعداد نقاط روی این خط عمودی نیز حداکثر این تعداد است.

پس همیشه می‌توان A ای پیدا کرد که $|A| \leq \sqrt{|S|}$ حالا رابطه بازگشتی

$$T(S) = T(S - A) + (|A|) \times (|A| - ۱)$$

را اگر باز کنیم به صورت مجموع مربعات یکسری عدد در می‌آید که مجموع خودشان برابر $|S| = n$ است. می‌دانیم در بدترین حالت این اعداد به صورت یکسری اعداد $\sqrt{n}$ هستند (برای اثبات آن می‌توانیم دو تا مجموعه که اندازه شان کمتر از این مقدار است را در نظر بگیریم با انتقال یک عضو از مجموعه کوچکتر به بزرگتر مجموع ثابت می‌ماند ولی مجموع مربعات زیاد می‌شود) پس در بدترین حالت جمع $\sqrt{n}$ تا عدد برابر $\sqrt{n}$ است که مربع‌هایشان برابر n می‌شود یعنی در کل مجموع مربعات حداکثر $n\sqrt{n}$ است.

سؤال ۴. [۱۶ نمره] به سؤالات زیر در مورد مرتب‌سازی درجی پاسخ دهید.

آ. [۶ نمره]

در یک آرایه یک‌نابه‌جایی برابر یک جفت جایگاه است که عدد داخل جایگاه اولی بزرگتر از عدد داخل جایگاه دومی باشد. برای مثال در یک آرایه مرتب‌شده تعداد نابه‌جایی‌ها برابر صفر است و در یک آرایه مرتب‌شده برعکس این تعداد برابر $\binom{n}{۲}$ است.

حال ثابت کنید اگر تعداد نابه‌جایی های یک آرایه dباشد. و الگوریتم insertion sort را روی آن اجرا کنیم با مرتبه $O(n + d)$ مرتب‌سازی انجام می‌شود.

ب. [۱۰ نمره]

حال فرض کنید یک آرایه نامرتب به ما داده‌اند به طوریکه هر کس با جای اصلی آن حداکثر ۱۰ جایگاه فاصله دارد.برای مثال عدد کمینه حداکثر در اندیس ۱۰ است. با ارائه روش مناسب برای مرتب‌سازی ثابت کنید $O(n)$ این کار ممکن است.

پاسخ: در Insertion sort یک حلقه n تایی داریم و به ازای هر اندیس به اندازه تعداد اعداد بزرگتر از خودش که قبل از او ظاهر شده‌اند پیمایش می‌کنیم که این تعداد دقیقا برابر نابه‌جایی هایی است که این اندیس می‌سازد. پس در مجموع به ازای هر اندیس این مقادیر را اگر جمع کنیم به ما مجموع تعداد نابه‌جایی ها را می‌دهد.

حالا برای بخش دوم ثابت می‌کنیم تعداد نابه‌جایی های دنباله‌ای با این شرط $O(n)$ است. برای اینکار صرفا کافیست بگوییم اگر عددی مثل $a > b$ قبل از b ظاهر شده باشد. $a - b \leq ۲۰$ چرا که در غیر این‌صورت یکی از a یا b با جایگاه اصلی‌شان بیش از ۱۰ واحد فاصله دارند. بنابراین تعداد نابه‌جایی‌ها حداکثر $۲۰n$ است و طبق بخش الف با اجرای مرتب‌سازی درجی الگوریتمی از $O(n + ۲۰n) = O(n)$ داریم.

سؤال ۵. [۱۶ نمره] فرض کنید nتا رشته داریم و می‌خواهیم رشته‌ها را به ترتیب الفبایی مرتب‌کنیم. رشته s در ترتیب الفبایی از p کوچکتر است اگر s یک پیشوند از p باشد یا اینکه اگر i کوچکترین مقداری باشد که $s_i \neq p_i$ در این‌صورت $s_i < p_i$ باشد.

برای مقایسه دو تا رشته باهم می‌توانیم روی تک تک کاراکترها از اول پیمایش کنیم و هر گاه به اولین جایگاهی رسیدیم که کاراکترهای متناظر نابرابرند آن کاراکتر را مقایسه کنیم.

حالا فرض کنید الگوریتم merge sort را روی این تعداد کاراکتر اجرا می‌کنیم. قبلا برای مقایسه هر دو عددی $O(۱)$ هزینه می‌دادیم. اکنون با استفاده از روشی که در بالا توضیح داده‌شد دو رشته را مقایسه می‌کنیم.

ثابت کنید در $O(Slogn)$ مرحله رشته‌ها مرتب می‌شوند که S برابر با جمع طول رشته‌های داده شده می‌باشد.

پاسخ: ثابت می‌کنیم ادغام دو تا آرایه از رشته‌ها مثل

$$s_۱, ..., s_k$$

و

$$p_۱, ..., p_l$$

با این روش ($O(\sum_{i=۱}^{k} |s_i| + \sum_{i=۱}^{l} |p_i|)$) طول می‌کشد. در مرحله اول ادغام $s_۱$ و $p_۱$ مقایسه می‌شوند که به اندازه ($O(min(|s_۱|, |p_۱|)$) طول می‌کشد. مستقل از اینکه هر کدام از اینها کوچکتر باشد اگر مجموع طول همه رشته‌ها را S بنامیم چیزی که باقی می‌ماند برابر $S - |s_۱|$ یا $S - |p_۱|$ است. اگر $T(s_۱, ..., s_k, p_۱, ..., p_l)$ زمان الگوریتم باشد داریم:

$$T(s_۱, ..., s_k, p_۱, ..., p_l) \leq min(|s_۱|, |p_۱|) + max(T(s_۲, ..., s_k, p_۱, ..., p_l), T(s_۱, ..., s_k, p_۲, ..., p_l)$$

که در هر صورت نشان می‌دهد: $T = O(S)$

پس می‌توان نتیجه گرفت هر لایه از درخت ادغام به اندازه $O(S)$ طول می‌کشد. می‌دانیم تعداد این لایه‌ها برابر $log_۲ n$ است بنابراین در کل این الگوریتم: $O(Slog_۲ n)$ طول می‌کشد.

۲ تمرینات عملی

سؤال ۶. [۱۲ نمره] محمد در یک مسابقه‌ی معلومات علمی شرکت کرده است. مسابقه شامل n سوال است. محمد سوال i ام را به احتمال p[i] درصد درست پاسخ می‌دهد و به احتمال 100 - p[i] درصد غلط پاسخ می‌دهد. هم چنین اگر محمد سوال i ام را درست پاسخ دهد، به اندازه‌ی c[i] تومان جایزه می‌گیرد. حال روند مسابقه بدین شکل است که سوال ها به ترتیب از محمد پرسیده می شود. هر بار اگر محمد سوال را درست پاسخ دهد، جایزه آن سوال را می‌گیرد و در صورت وجود، به سوال بعد می‌رود. اما اگر یک سوال را غلط پاسخ دهد، مسابقه تمام می شود و محمد جایزه هایی که تا کنون جمع کرده است را به خانه می‌برد. حال ما می‌توانیم برای کمک به محمد، ترتیب سوال ها را تغییر دهیم. هدف این است که پس از تغییر ترتیب سوال ها، امیدریاضی جایزه‌ای که محمد می‌برد ماکسیمم شود. حال محمد از شما پرسیده است که به ازای همه‌ی ترتیب های ممکن، ماکسیمم امیدریاضی جایزه چقدر است ؟

ورودی:
ابتدا در یک خط تعداد سوالات یعنی n می‌آید.
سپس n خط می‌آید که هر کدام شامل دو عدد c[i] و p[i] است.
خروجی:
در تنها سطر خروجی، عدد خواسته شده را با دقت ۳ رقم اعشار چاپ نمایید.
محدودیت‌ها:
$۱ \leq n \leq ۱۰^۵$
$۰ \leq p[i] \leq ۱۰۰$
$۰ \leq c[i] \leq ۱۰^۴$

ورودی و خروجی نمونه:

تست اول

stdin	**stdout**
4 100 1 80 1 66 1 20 1	2.434

تست دوم

stdin	stdout
3 50 1 33 6 70 1	2.326

پاسخ: برای حل این سوال، فرض می‌کنیم که ترتیبی از سوال ها بیشترین امیدریاضی جایزه را داشته باشد. واضح است سوالاتی که حتما درست پاسخ می‌دهیم را در ابتدای ترتیب قرار دهیم. ترتیب بین آنها نیز اهمیتی ندارد. حال این سوالات را از مجموعه‌ی سوالات حذف می‌کنیم و از این جا به بعد بقیه سوال‌ها را در نظر می‌گیریم. فرض می‌کنیم سوال i_ام در این ترتیب، حایزه‌ای برابر c_i داشته باشد و احتمال پاسخ‌دهی درست به آن نیز p_i باشد. حال دو سوال پشت سر هم i_ام و i+1_ام را در نظر بگیرید. ترتیب جدیدی از سوالات را در نظر بگیرید که در آن، جای این دو سوال عوض شده باشد. می‌دانیم امیدریاضی جایزه‌ی بدست آمده حداکثر برابر با حالت قبل است. با نوشتن امیدریاضی در دو حالت و لحاظ کردن کوچکتر مساوی بودن امیدریاضی حالت دوم بدست می‌آوریم که :

$$p_{i+1} * (1 - p_i) * c_{i+1} \leq p_i * (1 - p_{i+1}) * c_i$$

حال دو طرف را بر $(1 - p_i) * (1 - p_{i+1})$ تقسیم می‌کنیم. در این صورت خواهیم داشت:

$$\frac{p_{i+1} * c_{i+1}}{1 - p_{i+1}} \leq \frac{p_i * c_i}{1 - p_i}$$

بنابراین بدست آورده ایم که در حالت بهینه ترتیب سوال‌ها، سوال ها بر حسب

$$\frac{p * c}{1 - p}$$

به صورت نزولی مرتب شده است. پس کافی است ما سوال ها را بر حسب رابطه بالا نزولی مرتب کنیم تا به حالت بهینه برسیم. اردر این راه نیز از اردر مرتب‌سازی خواهد بود که اگر از مرتب‌سازی ادغامی استفاده کنیم، از $O(nlogn)$ خواهد بود. در زیر کد این سوال را که توسط آقای میکائیل قربانی زده شده است می‌بینید:

```
array = []
n = int(input())
for i in range(0, n):
    possibility, bonus = [int(value) for value in input().split()]
    if possibility == 0 or bonus == 0:
        continue
    possibility /= 100
    array.append(((1 - possibility) / (possibility * bonus), possibility, bonus))

array.sort()
prefixPossibility = 1
answer = 0
```

```
for (sort_value, possibility, bonus) in array:
    prefixPossibility *= possibility
    answer += prefixPossibility * bonus

print('%.3f'% answer)
```

سؤال ۷. [۱۲ نمره] محمد معلم کلاسی متشکل از n دانش‌آموز است. محمد بچه های کلاس را کنار هم در یک صف قرار می‌دهد. او که شدیدا به نظم اهمیت می‌دهد، می‌خواهد بچه ها را بر حسب قدشان از چپ به راست، به شکل نزولی مرتب کند. برای این کار هر بار می‌تواند دو دانش‌آموز مجاور را با هم جا به جا کند. اما او که خیلی کار دارد و حسابی خسته است، می‌خواهد با کمترین تعداد حرکات این کار را انجام دهد. برای این کار هم از شما خواسته است که مینیمم تعداد حرکات را بدست آورید.
دقت کنید که در این سوال از کد های آماده سایت‌ها و توابع مرتب سازی آماده نمی‌توانید استفاده کنید. هم چنین باید از مباحث آموخته شده در درس استفاده کنید.

ورودی:
در خط اول عدد n می‌آید که تعداد دانش‌آموز هاست. در خط بعدی n عدد a[i] پشت سر هم می‌آید که نشان دهنده‌ی قد افراد کلاس است.
خروجی:
یک عدد خروجی دهید که نشان‌دهنده‌ی کمترین تعداد حرکات لازم است.
محدودیت‌ها:
$۱ \leq n \leq ۱۰^۵$
$۰ \leq a[i] \leq ۱۰^۹$

ورودی و خروجی نمونه:

تست اول

stdin	stdout
4 0 3 4 2	4

تست دوم

stdin	stdout
5 5 2 4 2 1	1

پاسخ: می‌خواهیم قدهای افراد را به صورت نزولی مرتب کنیم. با جابه‌جا کردن دو نفر، تعداد نابه‌جایی هایی که قد افراد می‌سازد حداکثر یکی تغییر می‌کند. پس می‌دانیم برای رسیدن به حالت نهایی، لازم است حداقل به اندازه‌ی اختلاف نابه‌جایی های حالت اولیه و حالت ثانویه هزینه کنیم. هم‌چنین راهی با این تعداد حرکت وجود دارد. زیرا تا وقتی به حالت نهایی نرسیده‌ایم، دو نفر کنار

هم هستند که قدشان نابه‌جایی نمی‌سازد و می‌توانیم این دو نفر را جابه‌جا کنیم و یکی به تعداد نابه‌جایی ها اضافه شود(درستی این فرض را می‌توانید به سادگی نشان دهید). برای بدست آوردن تعداد نابه‌جایی ها نیز می‌توانیم از مرتب‌سازی ادغامی استفاده کنیم. بدین شکل که به صورت بازگشتی تعداد نابه‌جایی ها در نیمه‌ی چپ و راست را بدست آوریم و در ادغام این دو قسمت، هنگامی که یک عدد از یک نیمه انتخاب می‌شود ، با توجه به تعداد اعدادی قبل‌تر که از نیمه‌ی دیگر انتخاب شده است، می‌فهمیم چند نابه‌جایی با اعداد نیمه‌ی دیگر می‌سازد. اردر این راه نیز اردر مرتب‌سازی ادغامی یعنی $O(nlogn)$ خواهد بود. در زیر کد این سوال را که توسط آقای علی جوانمرد زده شده است می‌بینید:

```
def merge_sort_slice(array):
    if (len(array) <= 1):
        return array
    m = len(array) // 2
    L = merge_sort_slice(array[:m])
    R = merge_sort_slice(array[m:])
    return list(merge(L, R))


def merge(L, R):
    global b
    l = r = 0
    while l < len(L) or r < len(R):
        if l < len(L) and (
                r >= len(R) or L[l] >= R[r]):
            yield L[l]
            l += 1
        else:
            b += len(L) - l
            yield R[r]
            r += 1


number = int(input())
b = 0
s = []
s = list(map(int, input().split()))
s = merge_sort_slice(s)
print(b)
```

موفق باشید